

Reviewer Pack — Fourier Variance Ratio of Violated-Clause Counter Bounds Tre...

Entry ee44e38271ee · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 16:59:57 UTC

Fourier Variance Ratio of Violated-Clause Counter Bounds Tree-Resolution Depth

Entry ID: ee44e38271ee **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 16:59:57 UTC

1. Conjecture

Field A (mathematical branch): Walsh-Hadamard Fourier analysis of integer-valued lattice functions on $\{0,1\}^n$ (Friedgut/Bourgain/KKL-style concentration applied to non-Boolean counters, distinct from the $\{-1,+1\}$ indicator regime) **Field B** (complexity object): Tree-like resolution refutation depth $d_T(F \vdash \perp)$ of small unsatisfiable 3-CNFs, equivalently the minimum height of any DPLL search-tree certifying UNSAT

Statement:

Let F be an unsatisfiable 3-CNF on n variables with m clauses, and let $k_F: \{0,1\}^n \rightarrow \mathbb{Z}_{\geq 0}$ be the violated-clauses counter $k_F(x) = \#\{j: x \notin C_j\}$, expanded as $k_F = \sum_S \hat{k}_F(S) \chi_S$ in the Walsh-Hadamard basis. Define the dimensionless Fourier variance ratio $R(F) = \text{Var}(k_F) / E[k_F]^2$ over uniform $x \in \{0,1\}^n$ (equivalently $\sum_{S \neq \emptyset} \hat{k}_F(S)^2$ divided by $\hat{k}_F(\emptyset)^2$). Conjecture: every UNSAT 3-CNF F with $E[k_F] \geq 1$ satisfies $d_T(F \vdash \perp) \geq \lceil \log_2(1/R(F)) \rceil + 1$; a single F with $d_T(F)$ strictly below this Fourier-concentration floor falsifies the claim.

Rationale (proposer's reasoning):

A small variance ratio $R(F)$ means k_F is sharply concentrated around its mean, so the Friedgut/Bourgain noise picture predicts that random restrictions of small radius almost surely preserve at least one violated clause, forcing every branch of any DPLL tree to descend at least $\Theta(\log(1/R))$ levels before some clause empties. The integer-counter Fourier spectrum captures this concentration directly through Plancherel on k_F^2 , bridging KKL-style spectral concentration with the proof-complexity primitive of search-problem decision trees in a manner distinct from prior Boolean-indicator influence bounds.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d1365230ea4c7b2a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across ~ 80 UNSAT 3-CNFs ($n \in \{8, 10, 12, 14\}$, $m/n \approx 4.26$), conjecture is SUPPORTED iff every instance satisfies $d_T(F) \geq \lceil \log_2(1/R(F)) \rceil + 1$ AND Spearman $\rho(d_T, -\log_2 R) \geq 0.3$; FALSIFIED if any single instance violates the bound (slack < 0).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Fourier analysis unsatisfiable CNF resolution lower bound - Walsh-Hadamard expansion satisfiability DPLL tree depth - violated clauses counter Fourier concentration proof complexity

Top relevant hits considered: - [http://arxiv.org/abs/0806.1148v2] Unsatisfiable CNF Formulas need many Conflicts - [http://arxiv.org/abs/0905.1587v5] Unsatisfiable Linear CNF Formulas Are Large and Complex - [http://arxiv.org/abs/2511.07739v3] A Lower Bound for the Fourier Entropy of Boolean Functions on the Biased Hypercube - [http://arxiv.org/abs/1508.06336v1] SPRIGHT: A Fast and Robust Framework for Sparse Walsh-Hadamard Transform - [http://arxiv.org/abs/2101.09528v1] Hard satisfiable formulas for DPLL algorithms using heuristics with small memory - [http://arxiv.org/abs/0804.4343v1] Feature Unification in TAG Derivation Trees - [http://arxiv.org/abs/0911.3482v5] Complexity of Networks (reprise) - [http://arxiv.org/abs/nlin/0101006v1] On Complexity and Emergence

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=180.1s

5.1 Generated Python source

```
import random
import math
import json

def generate_unsat_cnf(n, m):
    literals = list(range(1, n + 1)) + [-l for l in range(1, n + 1)]
    clauses = set()
    while len(clauses) < m:
        clause = []
        for _ in range(3):
            l = random.choice(literals)
            if -l not in clause and l not in clause:
                clause.append(l)
        if any(l in clause for clause in clauses):
            continue
        clauses.add(tuple(sorted(clause)))
    return list(clauses)
```

```

def walsh_hadamard_transform(k_F):
    n = len(k_F)
    k_F.extend([0] * (2**n - len(k_F)))
    for s in range(1, n + 1):
        mask = (1 << s) - 1
        for i in range(2**(n-s)):
            j = i ^ mask
            k_F[i] += k_F[j]
            k_F[j] -= k_F[i]
    return [x / 2**s for s in range(n)]

def compute_fourier_variance_ratio(k_F):
    mean_k_F = sum(k_F) / len(k_F)
    variance = sum((k - mean_k_F)**2 for k in k_F) / len(k_F)
    k_F_hat = walsh_hadamard_transform(k_F)
    return (sum(k**2 for k in k_F_hat[1:])) / k_F_hat[0]**2

def compute_tree_resolution_depth(F):
    n = int(math.log2(len(F)))
    memo = {}

    def dp(partial_assignment):
        hash_value = hash(tuple(sorted(partial_assignment.items())))
        if hash_value in memo:
            return memo[hash_value]
        empty_clause = any(l not in partial_assignment for clause in F)
        if empty_clause:
            return 0
        max_depth = 0
        for l, value in partial_assignment.items():
            new_assignment = partial_assignment.copy()
            new_assignment[l] = 1 - value
            depth = dp(new_assignment) + 1
            if depth > max_depth:
                max_depth = depth
        memo[hash_value] = max_depth
        return max_depth

    return dp({})

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [8, 10, 12, 14]
    m_over_n = 4.26
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        m = int(m_over_n * n)
        F = generate_unsat_cnf(n, m)

        k_F = [sum(1 for clause in F if l not in clause) for l in range(1, 2*n + 1)]
        R_F = compute_fourier_variance_ratio(k_F)

```

```

d_T_F = compute_tree_resolution_depth(F)

instances_tested += 1

expected_bound = math.ceil(math.log2(1 / R_F)) + 1
if d_T_F < expected_bound:
    conjecture_holds = False
    counterexample = f"n={n}, m={m}, d_T(F)={d_T_F}, expected_bound={expected_bound}"

return {
    "metric_name": "tree_resolution_depth",
    "metric_value": d_T_F,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 180s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out after 180s (returncode=124) without producing any RESULT line or data, so neither the support nor the falsification condition can be evaluated. | next: Reduce the instance set or n-range (e.g., $n \in \{8,10\}$) and/or replace exact tree-resolution depth computation with a faster solver/bound, then rerun with a longer timeout to obtain support_fraction and Spearman ρ .

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	20939
2	propose	claude_max	opus	0	0	157348
3	preregistration	claude_max	opus	0	0	6725
4	novelty	claude_max	opus	0	0	3274
5	novelty	claude_max	opus	0	0	8933
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11402
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12277
8	judge	claude_max	opus	0	0	6750

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 227648 ms total latency. Provider mix: {'claude_max': 6, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/ee44e38271ee.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ee44e38271ee.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ee44e38271ee.tar.gz` (if generated)